计算机组成原理实验报告

计算机学院 吴文晖 25325278

一、实验一：译码器设计与仿真

1.实验原理

译码器是一种常见的组合逻辑电路，其主要功能是将输入的二进制编码转换为对应的输出信号。本实验设计一个 3-8 译码器，包含 3 位输入信号 A[2:0] 和 8 位输出信号 Y[7:0]。

当输入信号从 000 变化到 111 时，译码器应依次产生对应的输出结果。根据本次仿真波形，输出采用高电平有效的独热码形式，即每次只有与输入编码对应的输出位为 1，其余输出位为 0。

译码器属于组合逻辑电路，输出由当前输入决定，不需要时钟信号触发。因此，当输入编码发生变化时，输出应随之更新。

2.仿真结果与分析

Project Summary | decoder_3to8.v

D:/FPGA_lab/lab1/decoder_3to8/decoder_3to8.srcs/sources_1/new/decoder_3to8.v

```
module decoder_3to8(
input wire [2:0] A,
output reg [7:0] Y
);
always @(*) begin
case (A)
3'b000: Y=8'b0000_0001;
3'b001: Y=8'b0000_0010;
3'b010: Y=8'b0000_0100;
3'b011: Y=8'b0000_1000;
3'b100: Y=8'b0001_0000;
3'b101: Y=8'b0010_0000;
3'b110: Y=8'b0100_0000;
3'b111: Y=8'b1000_0000;
default: Y=8'b0000_0000;
endcase
end
endmodule
```

Project Summary | decoder_3to8.v | decoder_3to8_tb.v

D:/FPGA_lab/lab1/decoder_3to8/decoder_3to8.srcs/sim_1/new/decoder_3to8_tb.v

```
`timescale 1ns / 1ps
module decoder_3to8_tb;
reg [2:0] A;
wire [7:0] Y;
decoder_3to8 uut(.A(A), .Y(Y));
initial begin
A=3'b000; #10;
A=3'b001; #10;
A=3'b010; #10;
A=3'b011; #10;
A=3'b100; #10;
A=3'b101; #10;
A=3'b110; #10;
A=3'b111; #10;
$finish;
end
endmodule
```

RTL 和 TB 代码截图



波形截图

从仿真波形可以看出，输入信号 A[2:0] 在 0–80 ns 范围内依次经历 000、001、010、011、100、101、110 和 111 八种状态。

与输入变化对应，输出信号 Y[7:0] 依次显示为 01、02、04、08、10、20、40 和 80。波形中的输出数值采用十六进制显示，例如 01 表示二进制 00000001，例如 80 表示二进制 10000000

将输入与输出进行对照可知：当输入为 000 时，输出为 00000001，最低位有效。当输入为 001 时，输出为 00000010，第 1 位有效。随着输入编码逐渐增大，输出有效位依次向高位移动。当输入为 111 时，输出为 10000000，最高位有效。

整个仿真过程中，输出与输入编码保持正确的对应关系，每种输入均只有一位输出有效，符合 3-8 译码器的独热码输出特性。波形中未观察到明显的不定态，说明本次测试所覆盖的译码逻辑工作正常。

3.实验结论

本实验完成了 3-8 译码器的 RTL 设计和行为级仿真。通过 testbench 对 8 种输入编码进行遍历，验证了输入编码与输出独热码之间的对应关系。

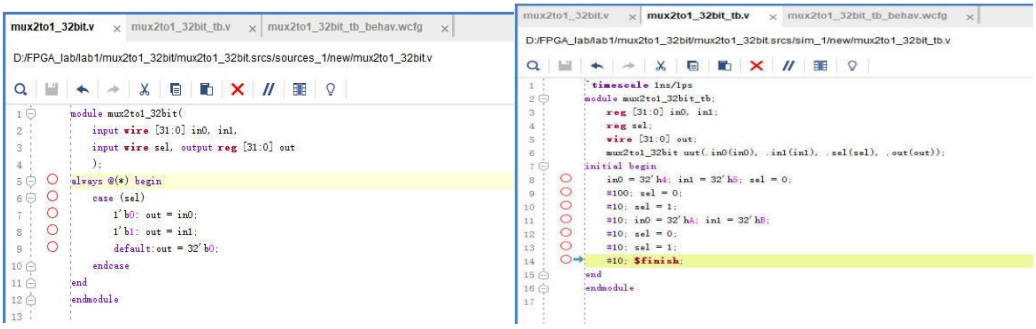
仿真结果表明，译码器能够根据输入信号产生正确的输出，8 种输入状态均符合预期。

二、实验二：32 位二选一数据选择器

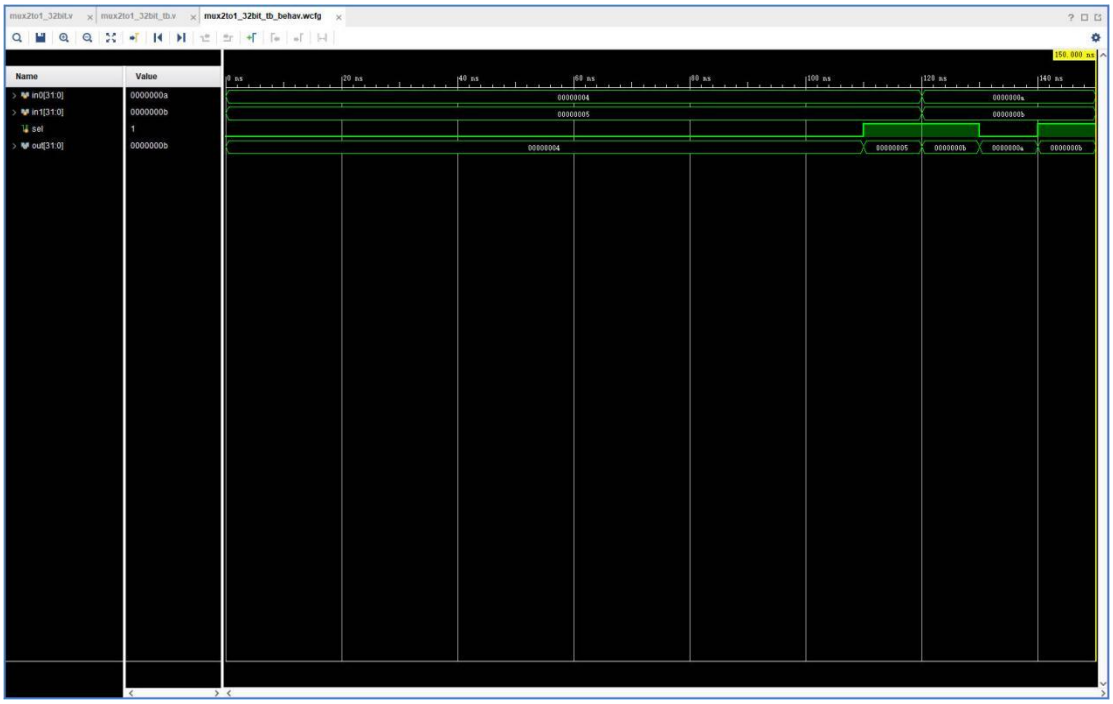
1.实验原理

32 位二选一数据选择器具有两组 32 位输入数据 in0、in1，一个选择控制信号 sel 和一个 32 位输出 out。其逻辑功能为：当 sel = 0 时，输出 out 等于输入 in0；当 sel = 1 时，输出 out 等于输入 in1。该电路属于组合逻辑电路，输出由当前输入数据和选择信号共同决定，不需要等待时钟沿触发。

2.仿真结果与分析



RTL 和 TB 代码截图



波形截图

从仿真波形可以观察到，输入数据 in0、in1、选择信号 sel 和输出数据 out 均按照测试平台设置发生变化。

在仿真阶段初始阶段，in0 的值为 00000004，in1 的值为 00000005。当 sel 为低电平时，输出 out 为 00000004，与 in0 一致，说明选择器正确选择了第一组输入数据。

当 sel 切换为高电平时，输出随之切换至第二组输入数据 00000005。随后，输入数据进行发生变化，输出也根据当前选择状态响应相应输入的变化。当选择信号再次改变时，输出切换至另一组输入数据。

仿真波形表明，输出数据能够按照选择信号在两组输入之间切换，且能够响应被选中输入数据的变化，符合 32 位二选一数据选择器的设计要求。

3.实验结论

本实验完成了 32 位二选一数据选择器的 RTL 设计和行为级仿真。通过观察输入数据、选择信号及输出数据的变化，验证了选择器的基本功能。

三、实验三：双向跑马灯设计

1.实验原理

跑马灯是一种通过周期性改变 LED 输出状态，实现灯光依次移动效果的数字电路。本实验使用 16 位 LED 输出信号 LED[15:0]，通过改变输出数据中有效位的位置，实现 LED 灯光的移动效果。

电路主要包含时钟信号 CLK、方向控制信号 SW0 和 LED 输出信号 LED[15:0]。

其中，时钟信号用于驱动电路状态更新，方向控制信号用于决定 LED 的移动方向。通过移位或状态更新操作，可以使 LED 图案向左或向右移动。当有效位移动到一端时，将其重新送回另一端，从而形成循环移动。

2.仿真结果与分析

Project Summary | running_light.v | running_light_tb.v

D:/FPGA_lab/lab1/running_light/running_light.srcs/sources_1/new/running_light.v

```
module running_light #(
    parameter integer STEP_TICKS = 10_000_000
)(
input wire CLK,
input wire SW0,
output reg [15:0] LED
);
reg [23:0] count = 0;
reg [3:0] pos = 4'd15;

always @(*) begin
    LED = 16'b1 << pos;
end

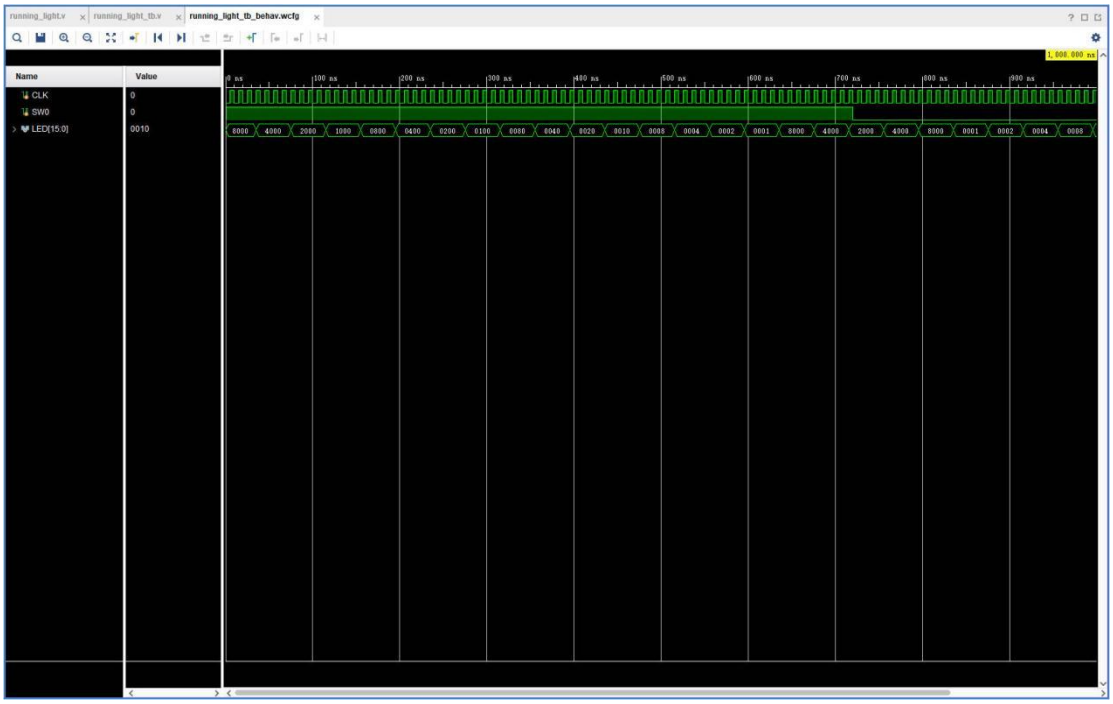
always @(posedge CLK) begin
    if(count == STEP_TICKS-1)begin
        count <= 0;
        if(SW0)begin
            pos <= (pos == 0) ? 4'd15:pos-1'b1;
        end else begin
            pos <= (pos == 15) ? 4'd0:pos+1'b1;
        end
    end else begin
        count <= count + 1'b1;
    end
end
endmodule
```

Project Summary | running_light.v | running_light_tb.v

D:/FPGA_lab/lab1/running_light/running_light.srcs/sim_1/new/running_light_tb.v

```
`timescale 1ns / 1ps
module running_light_tb;
    reg CLK = 0;
    reg SW0 = 1;
    wire [15:0] LED;
    always #5 CLK = ~CLK;
    running_light #(.STEP_TICKS(4)) uut(
        .CLK(CLK), .SW0(SW0), .LED(LED));
    initial begin
        #720; SW0 = 0;
        #720; $finish;
    end
endmodule
```

RTL 和 TB 代码截图



波形截图

从仿真波形可以观察到，CLK 持续产生周期性变化，作为跑马灯状态更新的时钟输入。SW0 为方向控制信号，LED[15:0] 则显示当前 LED 图案。

在仿真初始阶段，LED 输出为 8000，即 16 位二进制数据 1000 0000 0000 0000，最高位有效。随后，LED 图案依次变化为：8000 → 4000 → 2000 → 1000 → 0800 → 0400 → 0200 → 0100 → 0080 → 0040 → 0020 → 0010 → 0008 → 0004 → 0002 → 0001

这一变化过程表明，有效位逐步从最高位向最低位移动，LED 图案按照一个方向依次移动。

当输出到达 0001 后，波形又出现 8000，说明有效位从最低位回绕至最高位，实现了循环移动。

随后，SW0 发生变化，LED 输出的移动方向也随之改变。例如，波形中出现了 8000 →

4000 → 2000 → 4000 → 8000 等状态变化，表明有效位能够在方向控制信号改变后向相反方向移动。

在方向切换过程中，LED 输出并未统一恢复为固定初始值，而是根据当前状态继续变化，体现了方向切换不复位的设计特点。波形中还可以观察到 0001 与 8000 之间的状态衔接，说明循环回绕逻辑能够正常工作。

综合分析，仿真结果展示了 LED 单灯移动、两端回绕和方向控制的基本行为，与跑马灯的设计目标一致。

3.实验结论

仿真波形还展示了方向控制信号改变后 LED 移动方向相应变化的过程，且 LED 图案能够从当前状态继续运行，而不是无条件复位。

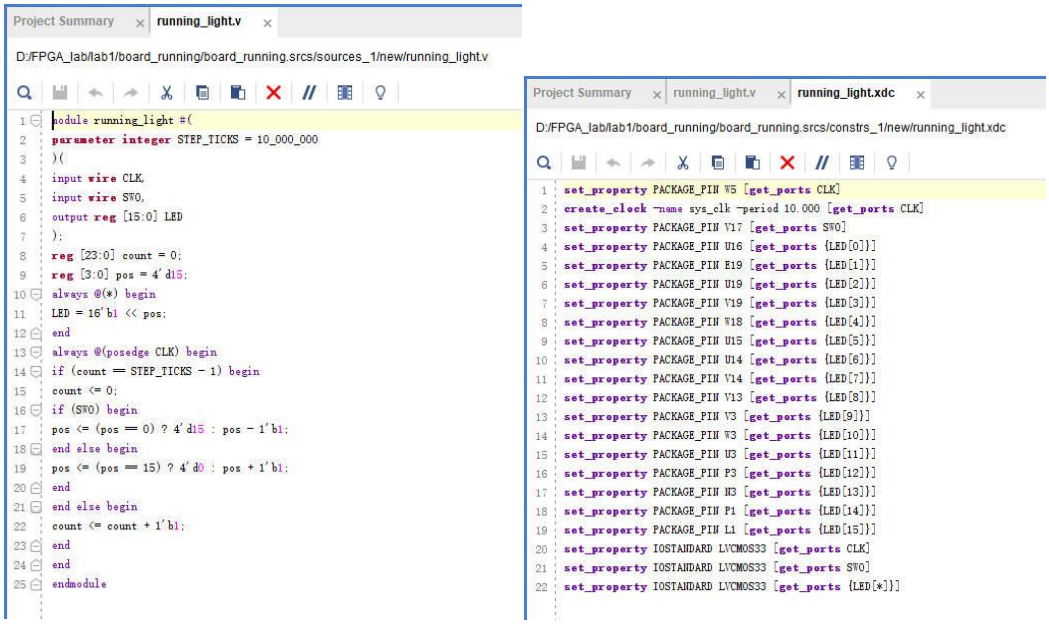
本实验完成了双向跑马灯的 RTL 设计和行为级仿真。通过观察 CLK、SW0 和 LED[15:0] 的变化，验证了 LED 图案能够依次移动，并在到达边界后循环回绕。

4.上板验证

在完成跑马灯模块的 RTL 设计和行为级仿真后，将设计下载至 FPGA 开发板，验证电路在实际硬件环境下的运行情况。通过观察开发板上的 LED 灯光变化，检查跑马灯的移动方向、循环回绕以及方向切换功能是否符合设计要求。

上板前，根据 FPGA 开发板的硬件原理图或用户手册，确定板载时钟输入、方向控制开关和 LED 输出端口对应的 FPGA 引脚，并在 XDC 约束文件中完成引脚分配。

随后，对设计进行综合和实现，检查是否存在影响下载的错误，并生成 FPGA 配置文件。将配置文件下载至开发板后，通过板载时钟和方向控制开关进行实际功能测试。



.v 和.xdc 文件

跑马灯视频：

https://github.com/shimmer256/computer-organization-lab/blob/main/lab1/borad_running/running_light.mp4

通过将跑马灯设计下载至 FPGA 开发板，对 LED 移动、两端回绕和方向切换功能进行了实际硬件验证。开发板上的 LED 显示与设计预期一致，说明 RTL 逻辑、引脚约束和硬件运行配置能够协同实现预定功能。

四、实验四：七段数码管扫描显示

1.实验原理

数码管是一种常用的数字显示器件，可通过控制不同的段选信号显示数字或部分字符。当需要使用多个数码管同时显示内容时，常采用动态扫描方式。

动态扫描的基本思想是：在某一时刻只选通一个数码管，同时输出该位所需要的段码；经过一段较短的时间后，再选通下一个数码管，并输出对应段码。通过快速循环扫描各个位选信号，人眼会感知到多个数码管同时稳定显示。

本实验涉及以下主要信号：Clk：时钟输入，用于驱动扫描状态更新。SW_in：模式控制输入，用于切换两种显示模式。display_out[10:0]：内部或模块输出的显示相关数据。AN[3:0]：四位数码管的位选控制信号。SEG[6:0]：七段数码管的段码输出信号。

在动态扫描过程中，位选信号决定当前哪一位数码管被选通，段码信号决定该位显示的数字。两者必须配合工作，才能正确显示多位数字。

2.仿真结果与分析

Project Summary | display_7seg.v | display_7seg_tb.v

D:/FPGA_lab/lab1/display_7seg/display_7seg.srcs/sources_1/new/display_7seg.v

```
module display_7seg #(
  parameter integer SCAN_TICKS = 50_000
)(
  input  wire       CLK,
  input  wire       SW_in,
  output reg  [10:0] display_out
);
localparam [6:0] SEG1 = 7'b100_1111;
localparam [6:0] SEG2 = 7'b001_0010;
localparam [6:0] SEG3 = 7'b000_0110;
localparam [6:0] SEG4 = 7'b100_1100;
reg [15:0] count = 0;
reg  [1:0] sel   = 0;

always @(*) begin
  case (sel)
    2'd0: display_out = {4'b1110,
                SW_in ? SEG4 : SEG1};
    2'd1: display_out = {4'b1101,
                SW_in ? SEG3 : SEG2};
    2'd2: display_out = {4'b1011,
                SW_in ? SEG2 : SEG3};
    2'd3: display_out = {4'b0111,
                SW_in ? SEG1 : SEG4};
    default: display_out = 11'b1111_1111111;
  endcase
end

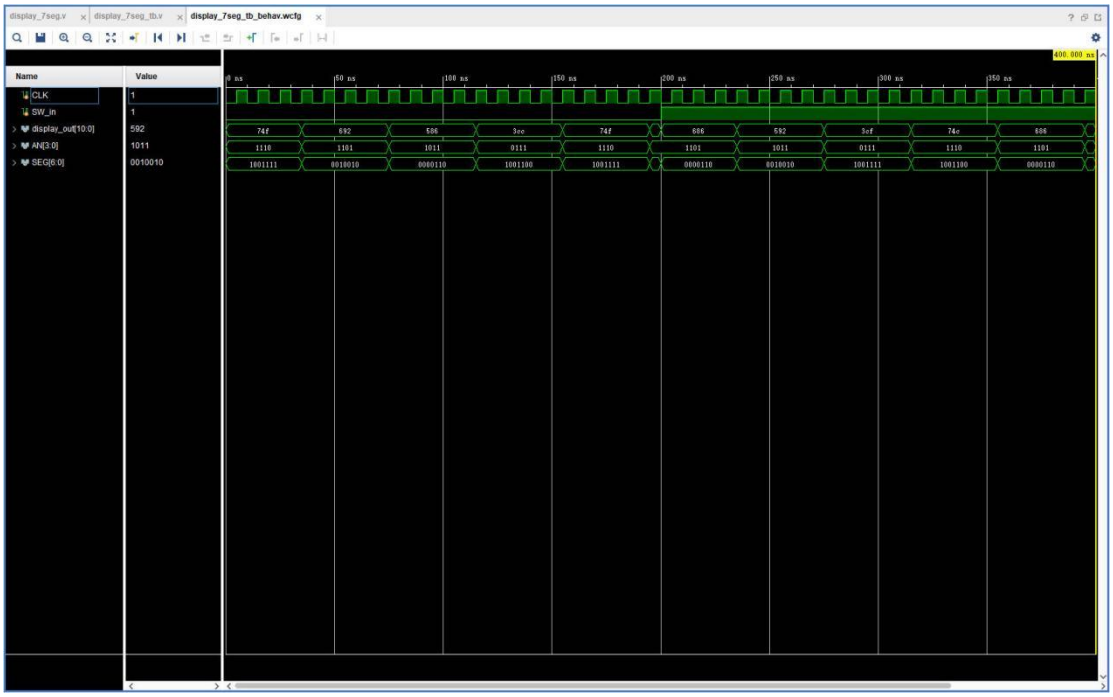
always @(posedge CLK) begin
  if (count == SCAN_TICKS - 1) begin
    count <= 0;
    sel   <= sel + 1'b1;
  end else begin
    count <= count + 1'b1;
  end
end
endmodule
```

Project Summary | display_7seg.v | display_7seg_tb.v

D:/FPGA_lab/lab1/display_7seg/display_7seg.srcs/sim_1/new/display_7seg_tb.v

```
`timescale 1ns / 1ps
module display_7seg_tb;
  reg CLK = 0, SW_in = 0;
  wire [10:0] display_out;
  wire [3:0] AN = display_out[10:7];
  wire [6:0] SEG = display_out[6:0];
  always #5 CLK = ~CLK;
  display_7seg #(.SCAN_TICKS(4)) uut(
    .CLK(CLK), .SW_in(SW_in),
    .display_out(display_out));
  initial begin
    #200; SW_in = 1;
    #200; $finish;
  end
endmodule
```

RTL 和 TB 代码截图



波形截图

从仿真波形可以观察到，CLK 持续产生周期性变化，为数码管扫描逻辑提供时钟。SW_in 用于控制显示模式，display_out[10:0]、AN[3:0] 和 SEG[6:0] 则反映显示数据、位选状态和段码输出的变化。

在本次仿真中，AN[3:0] 依次出现 1110、1101、1011 和 0111 等状态。根据常见的低电平有效位选方式，这些状态分别表示四个数码管中的不同一位被选通。位选信号持续轮换，说明扫描控制逻辑能够依次选择四个显示位置。

与此同时，SEG[6:0] 随着扫描状态和显示数据发生变化。例如，波形中可以观察到段码值 1001111、0010010、0000110 等不同编码，表明模块能够根据当前显示内容输出不同的段码组合。

display_out[10:0] 也在仿真过程中出现不同的数值，说明显示相关数据随当前模式或扫描过程发生变化。通过将显示数据、位选信号和段码输出结合起来，可以进一步核对每一位数码管所对应的显示内容。

从整体波形来看，四个位选信号能够轮流切换，段码信号能够随显示状态更新，符合动态扫描显示的基本工作原理。

3.实验结论

本实验完成了数码管动态扫描显示模块的 RTL 设计和行为级仿真。通过观察时钟信号、模式控制信号、显示数据、四个位选信号和七段段码输出，分析了动态扫描显示的工作过程。

仿真结果表明，四个位选信号能够依次切换，段码输出能够随扫描状态和显示数据发生变化，体现了动态扫描显示的基本特征。

4.上板验证

在完成数码管扫描显示模块的 RTL 设计和行为级仿真后，将设计下载至 FPGA 开发板，观察实际数码管的显示效果，验证四位数码管动态扫描、段码译码和两种显示模式切换功能。

通过实际硬件测试，进一步确认位选信号与段码输出能够正确配合，并检查显示是否存在乱码、缺位或扫描异常等问题。

上板前，根据开发板的原理图或用户手册，确认板载时钟、模式控制开关、四位数码管位选端口和七段段码端口对应的 FPGA 引脚。

在 XDC 文件中完成相应引脚约束，并设置开发板要求的 I/O 电平标准。完成综合和实现后，生成配置文件并下载至 FPGA 开发板。

下载完成后，观察数码管显示内容，并操作模式控制开关，检查不同模式下的显示结果是否符合设计要求。

Project Summary | display_7seg.v

D:/FPGA_lab/lab1/board_7seg/board_7seg.srcs/sources_1/new/display_7seg.v

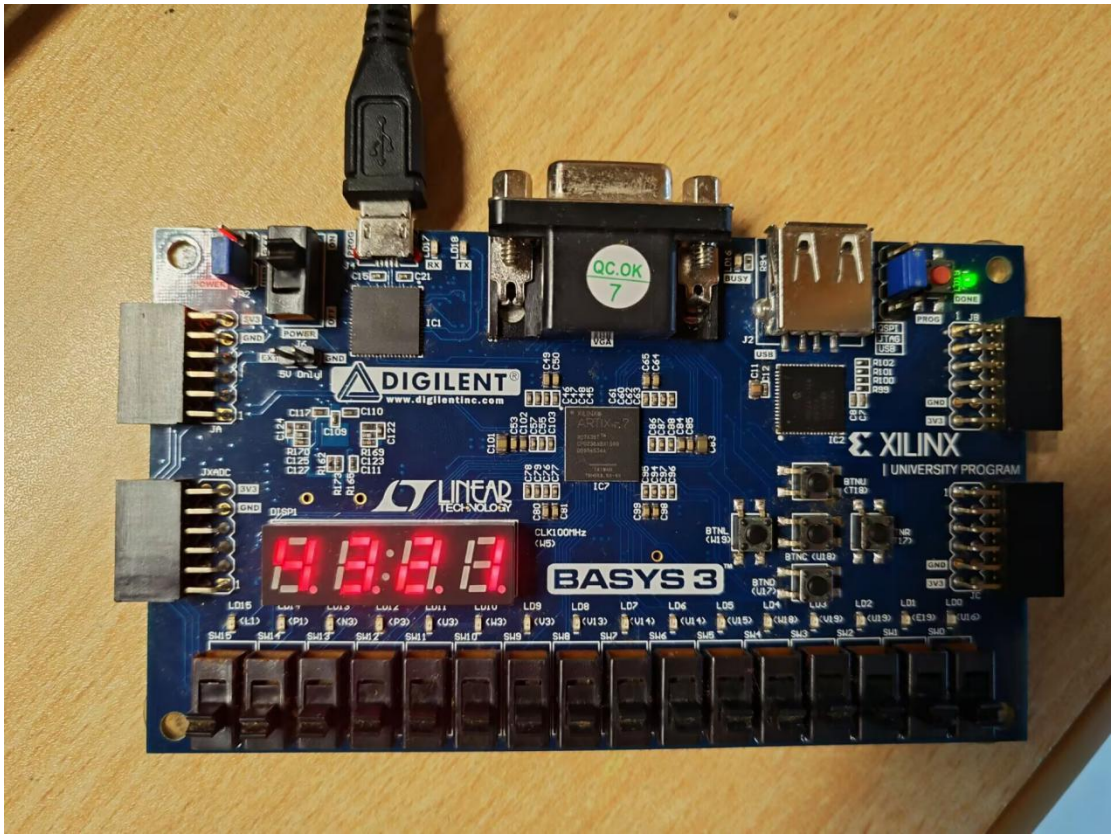
```
module display_7seg #(
parameter integer SCAN_TICKS = 50_000
)(
input wire CLK,
input wire SW_in,
output reg [10:0] display_out
);
localparam [6:0] SEG1 = 7'b100_1111;
localparam [6:0] SEG2 = 7'b001_0010;
localparam [6:0] SEG3 = 7'b000_0110;
localparam [6:0] SEG4 = 7'b100_1100;
reg [15:0] count = 0;
reg [1:0] sel = 0;
always @(*) begin
case (sel)
2'd0: display_out = {4'b1110,
SW_in ? SEG4 : SEG1};
2'd1: display_out = {4'b1101,
SW_in ? SEG3 : SEG2};
2'd2: display_out = {4'b1011,
SW_in ? SEG2 : SEG3};
2'd3: display_out = {4'b0111,
SW_in ? SEG1 : SEG4};
default: display_out = 11'b1111_1111111;
endcase
end
always @(posedge CLK) begin
if (count == SCAN_TICKS - 1) begin
count <= 0;
sel <= sel + 1'b1;
end else begin
count <= count + 1'b1;
end
end
endmodule
```

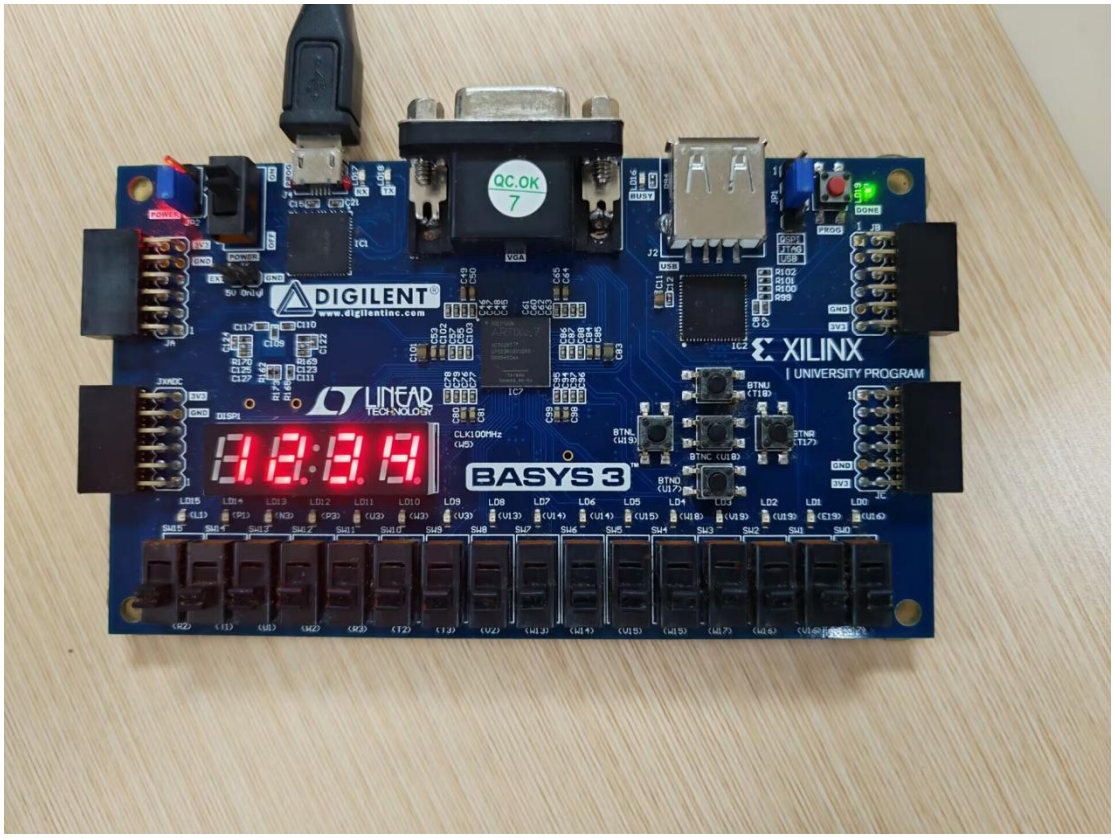
Project Summary | display_7seg.v | display_7seg.xdc

D:/FPGA_lab/lab1/board_7seg/board_7seg.srcs/constrs_1/new/display_7seg.xdc

```
set_property PACKAGE_PIN W5 [get_ports CLK]
set_property PACKAGE_PIN V17 [get_ports SW_in]
create_clock -name sys_clk -period 10.000 [get_ports CLK]
set_property PACKAGE_PIN W4 [get_ports {display_out[10]}]
set_property PACKAGE_PIN V4 [get_ports {display_out[9]}]
set_property PACKAGE_PIN U4 [get_ports {display_out[8]}]
set_property PACKAGE_PIN U2 [get_ports {display_out[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports CLK]
set_property IOSTANDARD LVCMOS33 [get_ports SW_in]
set_property IOSTANDARD LVCMOS33 [get_ports {display_out[*]}]
set_property PACKAGE_PIN W7 [get_ports {display_out[6]}]
set_property PACKAGE_PIN W6 [get_ports {display_out[5]}]
set_property PACKAGE_PIN U8 [get_ports {display_out[4]}]
set_property PACKAGE_PIN V8 [get_ports {display_out[3]}]
set_property PACKAGE_PIN U5 [get_ports {display_out[2]}]
set_property PACKAGE_PIN V5 [get_ports {display_out[1]}]
set_property PACKAGE_PIN U7 [get_ports {display_out[0]}]
# display_out[6:0] = CA, CB, CC, CD, CE, CF, CG
```

.v 和.xdc 文件





数码管显示照片

实际显示结果与预期一致，则说明设计不仅通过了行为级仿真，也能够在 FPGA 硬件环境中实现预定的动态扫描显示功能。